

CITIS Education Platform – Development Blueprint

1. Product Architecture

Platform Layers

CITIS EDUCATION PLATFORM

|

|— EXPERIENCE LAYER

| |— Super Admin Portal

| |— Institution Admin Portal

| |— Teacher Portal

| |— Student Portal

| |— Parent Portal

| |— Mobile Applications

|

|— CORE PLATFORM SERVICES

| |— Authentication & Identity

| |— Role-Based Access Control

| |— Institution / Campus Management

| |— Academic Year & Session Management

| |— Notification Engine

| |— Document Management

| |— Workflow Engine

| |— Audit Logs

| |— Reporting & Analytics

|

|— FUNCTIONAL MODULES

- | |— Admission
- | |— Fee Management
- | |— Attendance
- | |— Academics
- | |— Lesson Update
- | |— Event Management
- | |— Task Management
- | |— SMS / Communication
- | |— Mobile App
- | |— Parent Login
- | |— Online Class
- | |— LMS
- | |— HRMS & Payroll
- | |— Finance & Accounts
- |

|— INTEGRATION LAYER

- |— Zoom
- |— Google Meet
- |— Microsoft Teams
- |— SMS Gateway
- |— WhatsApp
- |— Payment Gateway
- |— Email
- |— Biometric Devices
- |— RFID / Smart Cards

└─ Accounting Systems

2. Recommended Technology Architecture

For development using **Claude Code or Codex**, I recommend a modern modular architecture.

Frontend

- **Next.js**
- React
- TypeScript
- Tailwind CSS
- Component library such as shadcn/ui
- React Query / TanStack Query
- React Hook Form
- Zod validation

Applications

apps/

└─ admin-web/

└─ teacher-web/

└─ student-web/

└─ parent-web/

└─ super-admin/

└─ mobile-app/

For mobile:

- React Native + Expo

Backend

Recommended

- **Node.js**
- TypeScript
- NestJS

NestJS is particularly suitable because the system will have many modules and enterprise workflows.

backend/

- └─ auth
- └─ institutions
- └─ users
- └─ admissions
- └─ academics
- └─ attendance
- └─ fees
- └─ lms
- └─ assessments
- └─ communication
- └─ events
- └─ tasks
- └─ hrms
- └─ finance
- └─ reports
- └─ integrations

Database

Primary database:

- **PostgreSQL**

Supporting services:

- Redis – caching, queues, sessions
- S3-compatible storage – documents, videos, assignments
- Elasticsearch/OpenSearch – optional advanced search
- Queue system – BullMQ or RabbitMQ

3. Multi-Tenant Architecture

CITIS should develop the platform as a **multi-tenant SaaS system**.

This is extremely important.

One CITIS platform should support:

CITIS

|

|— Institution A

| |— Campus 1

| |— Campus 2

|

|— Institution B

|

|— School Group C

| |— School 1

| |— School 2

| |— School 3

|

|— University D

Every institution should have:

- Institution profile

- Logo
- Branding
- Academic structure
- Users
- Students
- Teachers
- Courses
- Fees
- Communication settings
- Module subscription

This architecture will later support:

- White-label deployment
- Franchise/channel partners
- Multiple schools
- School groups
- Universities
- International institutions

4. User Roles

Platform-Level Roles

1. CITIS Super Admin

Controls:

- All institutions
- Subscription
- Module activation
- Platform configuration

- Support
- Analytics
- System health

2. CITIS Support/Admin

- Institution support
- User assistance
- Issue tracking
- Limited institution access

Institution-Level Roles

Institution Administrator

|

└— Principal / Director

└— Academic Administrator

└— Admission Manager

└— Finance Manager

└— HR Manager

└— Teacher

└— Class Teacher

└— Subject Teacher

└— Student

└— Parent

└— Staff

Each role should use **Role-Based Access Control (RBAC)**.

Do not hardcode permissions.

Use:

Role



Permission Group



Module Permission



Action Permission

Example:

Attendance

├— View

├— Create

├— Edit

├— Approve

├— Lock

└— Export

5. Core Modules Blueprint

Module 1: Admission Management

Features

- Online application
- Admission enquiry
- Lead capture
- Application form builder
- Document upload
- Application tracking

- Admission workflow
- Entrance assessment
- Interview scheduling
- Selection process
- Offer letter
- Admission confirmation
- Student creation
- Parent creation
- Fee payment integration

Workflow

Enquiry



Application



Document Submission



Application Review



Assessment / Interview



Selection



Offer



Fee Payment



Student Enrollment

6. Fee Management

Features

- Fee structure
- Class-wise fee
- Student-wise concessions
- Scholarships
- Installments
- One-time charges
- Recurring charges
- Late fees
- Discounts
- Tax configuration
- Online payment
- Payment receipts
- Refunds
- Outstanding tracking

Core Data Structure

Fee Category

Fee Structure

Student Fee Assignment

Invoice

Payment

Receipt

Discount

Scholarship

Refund

Integrations

- Razorpay
- PayU
- Stripe for international markets

7. Attendance Management

Attendance should support:

- Daily attendance
- Period attendance
- Teacher attendance
- Staff attendance
- Biometric integration
- RFID integration
- QR attendance
- Mobile attendance

Workflow

Institution



Academic Year



Class



Section



Student



Attendance Record

Dashboards

- Daily attendance %
- Class-wise attendance
- Student absenteeism
- Chronic absentee alerts
- Parent notifications

8. Academics Module

This is the academic backbone.

Core Features

- Academic year
- Terms
- Classes
- Sections
- Subjects
- Departments
- Teachers
- Timetable
- Class allocation
- Subject allocation

Academic Structure

Institution



Academic Year



Grade / Class



Section



Subject



Teacher



Students

For universities, the same engine can evolve into:

Institution



Faculty



Department



Programme



Semester



Course



Faculty



Students

This makes the platform suitable for both **schools and higher education**.

9. Lesson Planning and Lesson Update

The image mentions **Lesson Update**. I recommend expanding this into a complete **Lesson Planning and Delivery Module**.

Features

- Curriculum mapping
- Annual academic plan
- Term plan
- Weekly lesson plan
- Daily lesson plan
- Learning objectives
- Teaching resources
- Homework
- Lesson completion status
- Teacher updates
- Principal monitoring

Example

Grade 8



Mathematics



Algebra



Chapter



Learning Objectives



Lesson Plan



Teaching Material



Homework



Assessment

10. Event Management

Features

- Event creation
- Calendar
- Registration
- Invitations
- Attendance
- Parent notifications
- Resource allocation
- Event tasks
- Photo/document uploads

Examples:

- Sports day
- Annual function
- Parent meeting

- Examination
- Workshops
- Webinars

11. Task Management

A common workflow engine should power tasks across the platform.

Task Types

- Personal task
- Teacher task
- Student task
- Administrative task
- Department task
- Event task

Features

- Assign
- Priority
- Deadline
- Status
- Comments
- Attachments
- Reminders
- Escalation

12. Communication Module

Instead of developing only an SMS module, develop a **Unified Communication Hub**.

Communication Hub

└— SMS

└— Email

└— Push Notification

└— WhatsApp

└— In-App Notification

└— Announcement

Features

- Individual communication
- Group communication
- Class communication
- Emergency alerts
- Scheduled notifications
- Templates
- Delivery reports

13. Parent Portal

The Parent Portal should provide:

Dashboard

- Child profile
- Attendance
- Fees
- Homework
- Assignments
- Timetable
- Examination results
- School notices

- Teacher communication
- Events
- Online classes

For multiple children:

Parent

└─ Child 1

└─ Child 2

└─ Child 3

Parents should be able to switch between children.

14. Student LMS – Core CITIS Learning Platform

This should become one of CITIS's flagship products.

Course Structure

Programme



Course



Module



Unit



Chapter



Lesson



Learning Resource

Learning resources may include:

- Video
- PDF
- Presentation
- Interactive content
- Quiz
- Assignment
- SCORM package
- External links

LMS Features

Course Management

- Course creation
- Course catalog
- Course enrollment
- Cohorts
- Batch management
- Instructor allocation

Learning Delivery

- Video learning
- Live classes
- Recorded classes
- Reading material
- Interactive learning
- Progress tracking

Assessment

- Quiz
- MCQ
- Descriptive answers
- Assignments
- Projects
- Practical submissions

Progress

Not Started



In Progress



Completed



Assessment



Certified

15. Online Class Module

Build this as an **integration layer**, not as a proprietary video-conferencing platform initially.

Integrations:

- Zoom
- Google Meet
- Microsoft Teams
- Cisco/Webex where required

Features

- Schedule class

- Invite students
- Automatic calendar entry
- Attendance
- Recording links
- Class chat
- Recording repository

16. Assessment and Examination Engine

Although not explicitly shown in the image, this is essential for the LMS.

Assessment Types

- Practice quiz
- Formative assessment
- Summative assessment
- Assignment
- Project
- Viva
- Practical

Question Bank

Subject



Topic



Difficulty



Question Type



Question

Question types:

- MCQ
- Multiple select
- True/False
- Fill in blanks
- Matching
- Short answer
- Long answer

Features

- Randomization
- Question pools
- Negative marking
- Time limits
- Attempts
- Auto evaluation
- Manual evaluation
- Result publication

17. HRMS and Payroll

This should remain modular.

HRMS

- Employee records
- Joining
- Documents
- Leave

- Attendance
- Performance
- Department
- Designation

Payroll

- Salary structure
- Earnings
- Deductions
- Payslip
- Tax
- Payroll processing

18. Finance and Accounts

This should integrate with Fee Management.

Core Features

- Chart of accounts
- Receivables
- Payables
- Expense management
- Budgeting
- Cost centres
- Financial reports

However, I recommend keeping this module as a **separate bounded domain** so that schools can alternatively integrate with Tally or another accounting platform.

19. Central Platform Services

These should be developed once and reused by every module.

Authentication

Support:

- Email/password
- Mobile OTP
- Google login
- Microsoft login
- SSO

Document Management

Central repository for:

- Student documents
- Teacher documents
- Admission documents
- Assignments
- Learning materials
- Certificates

Notification Engine

Every module should publish events.

Example:

Student Absent



Event Generated



Notification Engine



SMS + Push Notification



Parent

Audit Logs

Track:

Who

What

When

Previous Value

New Value

IP / Device

This is critical for enterprise clients.

20. Suggested Database Design

A simplified high-level model:

Tenant

|

├— Institution

|

├— Campus

|

├— User

|

├— Role

|

|— Permission

|

|— Student

|

|— Parent

|

|— Employee

|

|— AcademicYear

|

|— Class

|

|— Section

|

|— Subject

|

|— Course

|

|— Module

|

|— Lesson

|

|— Assessment

|

|— Attendance

|

|— FeeStructure

|

|— Invoice

|

|— Payment

|

|— Event

|

|— Task

|

└ Notification

Every major table should include:

id

tenant_id

institution_id

created_at

updated_at

created_by

updated_by

status

21. Recommended API Strategy

Use REST APIs initially.

Example:

/api/v1/auth

/api/v1/users

/api/v1/institutions

/api/v1/students

/api/v1/admissions

/api/v1/academics

/api/v1/attendance

/api/v1/fees

/api/v1/courses

/api/v1/lessons

/api/v1/assessments

/api/v1/events

/api/v1/tasks

/api/v1/notifications

Document APIs using:

- OpenAPI
- Swagger

This will make it easier for Claude/Codex to work consistently across modules.

22. Recommended Development Approach

Do Not Ask Claude/Codex to Build the Entire System at Once

Instead, development should happen in controlled phases.

Phase 0 – Foundation

Build first:

- Repository architecture
- Authentication

- Multi-tenancy
- RBAC
- Institution setup
- User management
- Design system
- API standards
- Database conventions
- Audit logging

Phase 1 – Core School Management

Admissions

Academics

Students

Parents

Attendance

Communication

Phase 2 – Financial Operations

Fee Management

Invoices

Payments

Receipts

Finance Integration

Phase 3 – LMS

Course Management

Content Management

Learning Delivery

Assignments

Assessments

Progress Tracking

Certificates

Phase 4 – Advanced Modules

Online Classes

Events

Task Management

HRMS

Payroll

Advanced Reporting

Phase 5 – Intelligence Layer

Future CITIS AI capabilities:

- AI lesson plan generation
- AI question generation
- AI student performance analysis
- At-risk student identification
- AI learning recommendations
- Teacher copilot
- Administrative copilot

23. Monorepo Structure for Claude/Codex

I strongly recommend this structure:

citis-education-platform/

|

└─ apps/

|

└─ super-admin/

|

└─ institution-admin/

|

└─ teacher-portal/

|

└─ student-portal/

|

└─ parent-portal/

|

└─ mobile/

|

└─ services/

|

└─ api/

|

└─ worker/

|

└─ notification-service/

|

└─ packages/

|

└─ ui/

|

└─ database/

|

└─ auth/

|

└─ types/

|

└─ config/

|

└─ sdk/

|

└─ docs/

```
| |└─ architecture/
| |└─ modules/
| |└─ api/
| |└─ database/
| └─ development/
|
|└─ infrastructure/
| |└─ docker/
| |└─ terraform/
| └─ kubernetes/
|
└─ CLAUDE.md
```

24. Critical File: CLAUDE.md

This will become the instruction manual for Claude Code.

It should define:

- Technology stack
- Architecture rules
- Naming conventions
- Database rules
- API conventions
- Security requirements
- Testing requirements
- Coding standards
- Module boundaries

Example principles:

1. Never create database tables without tenant isolation.
2. Never bypass RBAC.
3. Every mutation must create an audit record.
4. Every API requires validation.
5. Business logic must not exist inside controllers.
6. Modules must not directly access another module's database tables.
7. Use events for cross-module communication.
8. All code must include automated tests.
9. Do not introduce dependencies without approval.
10. Preserve backward API compatibility.

This is extremely important if multiple AI coding sessions will be used.

25. Recommended AI Coding Workflow

Step 1 – Create Product Requirements

Create one document per module.

Example:

`/docs/modules/admission.md`

/docs/modules/attendance.md

/docs/modules/lms.md

Each document should contain:

Purpose

Users

User Stories

Business Rules

Screens

Database Entities

API Requirements

Permissions

Integrations

Acceptance Criteria

Step 2 – Ask Claude/Codex for One Module

Example:

Read the repository architecture and CLAUDE.md. Implement the Admission Management module only. Do not modify unrelated modules. Follow the documented database, API and RBAC conventions. First create an implementation plan and list database changes. Wait for approval before applying migrations.

Step 3 – Generate Tests

After every module:

Unit Tests

Integration Tests

API Tests

Permission Tests

26. Development Priorities

My recommended **MVP for CITIS** would be:

Release 1

- Institution setup
- User management
- RBAC
- Students
- Parents
- Academics
- Attendance
- Communication

Release 2

- Fee Management
- Online payments
- Mobile app
- Parent portal

Release 3

- LMS
- Courses
- Content
- Assignments
- Assessments
- Certificates

Release 4

- Online classes

- HRMS
- Payroll
- Finance

CITIS EduSphere

An Integrated Digital Education and Learning Management Ecosystem

The architecture should support three deployment models:

SCHOOL LMS

+

SCHOOL ERP

+

HIGHER EDUCATION LMS

+

CORPORATE LEARNING

The most important architectural decision is to build **one common platform with independently deployable modules**, rather than building separate products for admissions, ERP and LMS.

That will make CITIS significantly more scalable and easier to commercialize.

CITIS Education Platform

Software Requirements Specification (SRS) and AI Development Master Blueprint

Version: 1.0

Product Owner: CITIS Infotech

Document Type: Master Product, Software and AI Development Specification

Platform Type: Multi-Tenant Education ERP + LMS + Digital Learning Ecosystem

1. Executive Summary

CITIS Infotech will develop a comprehensive, cloud-based, modular Education Management Platform combining:

- School ERP
- Learning Management System
- Student Information System
- Parent Engagement Platform
- Teacher Productivity Tools
- Digital Learning
- Assessment and Examination Management
- Communication Platform
- Mobile Applications
- HRMS and Payroll
- Finance and Accounts

The platform shall be designed as a **multi-tenant SaaS product**, allowing CITIS to serve:

- Individual schools
- School chains
- School groups
- Colleges
- Universities
- Training institutions
- Skill development organizations
- Corporate learning programs

The architecture shall support modular activation, allowing institutions to use only the modules they subscribe to.

The product shall be developed using AI-assisted coding tools such as:

- Claude Code
- OpenAI Codex

All development must follow the architecture, coding standards, database rules and module boundaries defined in this document.

2. Product Vision

Vision Statement

To build a unified digital ecosystem that enables educational institutions to manage administration, academics, learning, assessments, communication and institutional operations through a single intelligent platform.

The platform shall eliminate fragmented systems and provide a single source of truth for all institutional data.

3. Product Architecture

CITIS EDUCATION PLATFORM

|

| — PLATFORM ADMINISTRATION

| | — CITIS Super Admin

| | — Tenant Management

| | — Subscription Management

| | — Module Management

| | — Platform Analytics

|

| — INSTITUTION MANAGEMENT

| | — Institution

| | — Campus

| | — Academic Configuration

| | — Branding

|

└─ CORE ERP

| └─ Admission

| └─ Student Information

| └─ Academics

| └─ Attendance

| └─ Fee Management

| └─ Communication

|

└─ LEARNING PLATFORM

| └─ LMS

| └─ Course Management

| └─ Content Management

| └─ Assignments

| └─ Assessments

| └─ Certificates

| └─ Learning Analytics

|

└─ ENGAGEMENT

| └─ Parent Portal

| └─ Student Portal

| └─ Teacher Portal

| └─ Events

| └─ Tasks

| └─ Notifications

|

└─ ADVANCED MODULES

| └─ Online Classes

| └─ HRMS

| └─ Payroll

| └─ Finance

|

└─ INTEGRATIONS

└─ Payment Gateway

└─ SMS

└─ WhatsApp

└─ Email

└─ Zoom

└─ Google Meet

└─ Microsoft Teams

└─ Biometric Devices

└─ Accounting Systems

4. Functional Modules

The platform shall contain the following modules.

Module	Release Priority
Platform Administration	Phase 0
Authentication	Phase 0
Institution Management	Phase 0
User & Role Management	Phase 0

Student Information System Phase 1

Admission Management	Phase 1
Academics	Phase 1
Attendance	Phase 1
Communication	Phase 1
Parent Portal	Phase 2
Fee Management	Phase 2
Mobile App	Phase 2
LMS	Phase 3
Assessments	Phase 3
Online Classes	Phase 4
Event Management	Phase 4
Task Management	Phase 4
HRMS	Phase 4
Payroll	Phase 4
Finance & Accounts	Phase 5
AI Intelligence Layer	Phase 6

5. User Roles

5.1 Platform Roles

CITIS Super Admin

Can:

- Create tenants
- Create institutions

- Activate modules
- Manage subscriptions
- Manage platform settings
- View platform analytics
- Manage support access
- Manage integrations

CITIS Platform Support

Can:

- View assigned institutions
- Access support tickets
- Diagnose system issues
- Access logs based on authorization
- Perform restricted administrative actions

5.2 Institution Roles

Institution Administrator

Full institutional administrative access.

Principal / Director

Access to:

- Institutional dashboard
- Academic analytics
- Teacher performance
- Student performance
- Attendance
- Reports

Admission Manager

Access to:

- Leads
- Applications
- Documents
- Admission workflow

Academic Administrator

Access to:

- Academic structure
- Classes
- Subjects
- Timetable
- Teachers
- Academic reports

Teacher

Access to:

- Assigned classes
- Students
- Attendance
- Lesson plans
- Learning content
- Assignments
- Assessments

Student

Access to:

- Courses
- Classes

- Assignments
- Assessments
- Results
- Attendance
- Certificates

Parent

Access only to linked children.

Finance Manager

Access to:

- Fees
- Payments
- Receipts
- Finance reports

HR Manager

Access to:

- Employees
- Leave
- Attendance
- Payroll

6. Role-Based Access Control

Permissions shall not be hardcoded.

The authorization hierarchy shall be:

User



Role

↓

Permission

↓

Module

↓

Resource

↓

Action

Actions shall include:

VIEW

CREATE

UPDATE

DELETE

APPROVE

REJECT

EXPORT

PUBLISH

ARCHIVE

Example:

attendance.view

attendance.create

attendance.update

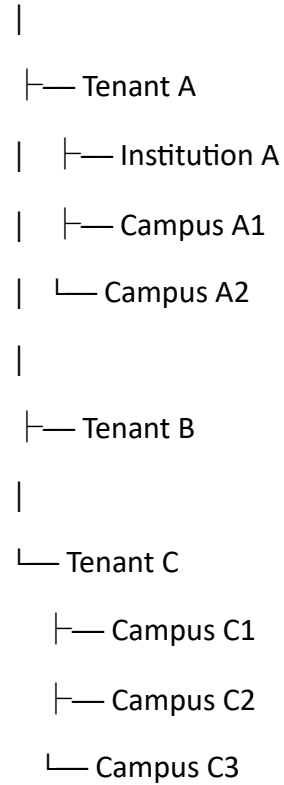
attendance.approve

attendance.export

7. Multi-Tenant Architecture

The system shall support multiple tenants.

CITIS PLATFORM



Every business record shall be tenant-aware.

Minimum fields:

id

tenant_id

institution_id

campus_id

created_at

updated_at

created_by

updated_by

status

No query shall access tenant data without tenant isolation.

8. Exact Repository Structure

The platform shall use a monorepo.

citis-education-platform/

|

└─ apps/

| |

| └─ super-admin/

| | └─ app/

| | └─ components/

| | └─ features/

| | └─ services/

| |

| └─ institution-admin/

| | └─ app/

| | └─ components/

| | └─ features/

| | └─ services/

| |

| └─ teacher-portal/

| └─ student-portal/

| └─ parent-portal/

| └─ mobile/

|

└─ services/

```
| |
| |─ api/
| | |─ src/
| | | |─ modules/
| | | | |─ auth/
| | | | |─ tenants/
| | | | |─ institutions/
| | | | |─ users/
| | | | |─ admissions/
| | | | |─ students/
| | | | |─ academics/
| | | | |─ attendance/
| | | | |─ fees/
| | | | |─ lms/
| | | | |─ assessments/
| | | | |─ events/
| | | | |─ tasks/
| | | | |─ communication/
| | | | |─ hrms/
| | | | |─ payroll/
| | | | |─ finance/
| | | | |─ reports/
| | | |
| | | |─ common/
| | | |─ config/
```



```
| | | |─ database/
| | | |─ guards/
| | | |─ middleware/
| | | |─ main.ts
| |
| |─ worker/
| | |─ notifications/
| | |─ reports/
| | |─ certificates/
| | |─ integrations/
| |
| |─ notification-service/
|
|─ packages/
| |
| |─ ui/
| |─ database/
| |─ auth/
| |─ types/
| |─ config/
| |─ validation/
| |─ sdk/
|
|─ docs/
| |
```

- | |— architecture/
- | |— modules/
- | |— database/
- | |— api/
- | |— user-stories/
- | |— screens/
- | |— development/
- |
- |— infrastructure/
- | |
- | |— docker/
- | |— terraform/
- | |— kubernetes/
- | |— ci-cd/
- |
- |— tests/
- | |— integration/
- | |— e2e/
- | |— performance/
- |
- |— CLAUDE.md
- |— CODEX.md
- |— README.md
- |— package.json

9. Technology Stack

Frontend

Next.js

React

TypeScript

Tailwind CSS

shadcn/ui

TanStack Query

React Hook Form

Zod

Mobile

React Native

Expo

TypeScript

Backend

Node.js

NestJS

TypeScript

Database

PostgreSQL

Redis

File Storage

AWS S3

or

S3-Compatible Object Storage

Background Processing

BullMQ

Redis Queues

API Documentation

OpenAPI

Swagger

10. Database Blueprint

10.1 Core Tenant Tables

tenants

id

name

slug

status

created_at

updated_at

institutions

id

tenant_id

name

institution_type

logo_url

email

phone

website

status

created_at

updated_at

campuses

id

tenant_id

institution_id

name

address

city

state

country

timezone

status

11. Identity and Access Tables

users

id

tenant_id

email

mobile

password_hash

first_name

last_name

profile_image

status

last_login_at

created_at

updated_at

roles

id

tenant_id

name

code

description

permissions

id

module

resource

action

code

user_roles

id

user_id

role_id

institution_id

campus_id

role_permissions

id

role_id

permission_id

12. Student Information System Database

students

id
tenant_id
institution_id
campus_id
user_id
student_code
first_name
middle_name
last_name
date_of_birth
gender
status
admission_date

parents

id
tenant_id
user_id
first_name
last_name
email
mobile

student_parents

id
student_id
parent_id
relationship

is_primary

enrollments

id

student_id

academic_year_id

class_id

section_id

roll_number

status

13. Academic Database Schema

academic_years

id

tenant_id

institution_id

name

start_date

end_date

status

terms

id

academic_year_id

name

start_date

end_date

grades

id

institution_id

name

sequence

classes

id

grade_id

name

academic_year_id

sections

id

class_id

name

capacity

subjects

id

institution_id

name

code

teacher_subject_assignments

id

teacher_id

subject_id

class_id

section_id

academic_year_id

14. LMS Database Schema

programmes

id

tenant_id

institution_id

name

code

description

status

courses

id

programme_id

title

code

description

thumbnail

status

course_modules

id

course_id

title

description

sequence

lessons

id

module_id

title

description

sequence

estimated_duration

learning_resources

id

lesson_id

resource_type

title

url

file_path

duration

sequence

Resource types:

VIDEO

PDF

DOCUMENT

PRESENTATION

LINK

SCORM

INTERACTIVE

15. Assessment Database

assessments

id

course_id
title
assessment_type
total_marks
passing_marks
duration_minutes
attempt_limit
status

questions

id
assessment_id
question_type
question_text
marks
difficulty

question_options

id
question_id
option_text
is_correct

assessment_attempts

id
assessment_id
student_id
started_at
submitted_at

score

status

16. API Standards

All APIs shall use:

/api/v1/

Example:

POST /api/v1/auth/login

POST /api/v1/auth/logout

GET /api/v1/students

POST /api/v1/students

GET /api/v1/students/:id

PATCH /api/v1/students/:id

GET /api/v1/courses

POST /api/v1/courses

GET /api/v1/courses/:id

PATCH /api/v1/courses/:id

17. Standard API Response

Success Response

```
{  
  "success": true,  
  "data": {},  
  "meta": {
```

```
    "requestId": "uuid"
  }
}
```

List Response

```
{
  "success": true,
  "data": [],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 100,
    "totalPages": 5
  }
}
```

Error Response

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request",
    "details": []
  }
}
```

18. API Rules

Every API shall:

- Require authentication unless explicitly public
- Validate input
- Enforce tenant isolation
- Enforce permissions
- Generate audit logs for mutations
- Use pagination for list APIs
- Support filtering where required
- Use UUID identifiers
- Return consistent error formats
- Include request tracing

No controller shall contain significant business logic.

Architecture:

Controller



Service



Domain Logic



Repository



Database

19. Event-Driven Architecture

Modules shall communicate through events.

Example:

Student Enrolled



Event Bus



- |— Create LMS Enrollment
- |— Send Parent Notification
- |— Generate Student Credentials
- └— Update Analytics

Standard event naming:

student.created

student.enrolled

attendance.marked

payment.completed

course.enrolled

assignment.submitted

assessment.completed

certificate.issued

20. Screen Inventory

Super Admin

1. Login
2. Dashboard
3. Tenant Management
4. Institution Management
5. Subscription Management
6. Module Activation
7. User Management

8. Platform Analytics
9. Support Management
10. Integration Management

Institution Administration

1. Dashboard
2. Institution Profile
3. Campus Management
4. Academic Year
5. User Management
6. Role Management
7. Student Directory
8. Parent Directory
9. Teacher Directory
10. Reports

Admission

1. Admission Dashboard
2. Lead List
3. Enquiry Form
4. Application List
5. Application Details
6. Document Review
7. Assessment Scheduling
8. Interview Scheduling
9. Admission Decision

10. Offer Letter

11. Enrollment Confirmation

Academics

1. Academic Dashboard
2. Academic Year
3. Grades
4. Classes
5. Sections
6. Subjects
7. Teacher Allocation
8. Timetable
9. Curriculum Mapping
10. Academic Reports

Attendance

1. Attendance Dashboard
2. Daily Attendance
3. Period Attendance
4. Attendance Reports
5. Absentee Alerts
6. Attendance Analytics

LMS

1. LMS Dashboard
2. Course Catalog

3. Course Builder
4. Module Builder
5. Lesson Builder
6. Content Upload
7. Student Enrollment
8. Learning Progress
9. Assignment Management
10. Assessment Management
11. Certificates
12. Learning Analytics

21. Core User Stories

Admission

US-ADM-001

As an admission manager, I want to capture a student enquiry so that I can track potential applicants.

Acceptance criteria:

- Enquiry form captures parent and student details.
- System generates unique enquiry ID.
- Enquiry can be assigned to a counselor.
- Follow-up activity can be scheduled.

US-ADM-002

As a parent, I want to submit an online admission application so that I can apply without visiting the institution.

Acceptance criteria:

- Parent can create an application.

- Required documents can be uploaded.
- Application progress can be tracked.
- Application confirmation is generated.

22. Attendance User Stories

US-ATT-001

As a teacher, I want to mark attendance for my assigned class so that student attendance records remain accurate.

Acceptance criteria:

- Teacher can access assigned classes only.
- Attendance can be marked present, absent or late.
- Attendance cannot be modified after locking without authorization.
- Parents receive absence alerts.

23. LMS User Stories

US-LMS-001

As an administrator, I want to create a course so that students can access structured learning content.

Acceptance criteria:

- Course can contain multiple modules.
- Modules can contain lessons.
- Lessons can contain multiple resources.
- Course visibility can be draft or published.

US-LMS-002

As a student, I want to track my course progress so that I know how much learning I have completed.

Acceptance criteria:

- Progress is calculated automatically.
- Completed lessons are recorded.
- Assessment completion is included.
- Dashboard displays overall progress.

24. AI Development Rules

The AI coding agent shall never:

- Create unrelated modules
- Change database conventions without approval
- Bypass authorization
- Hardcode tenant IDs
- Hardcode permissions
- Access cross-tenant data
- Introduce dependencies without justification
- Modify unrelated files unnecessarily

25. CLAUDE.md Master Instructions

CITIS Education Platform Development Rules

You are developing an enterprise multi-tenant education platform.

Mandatory rules:

1. Use TypeScript.
2. Follow existing module boundaries.

3. Every database entity must enforce tenant isolation where applicable.
4. Never bypass RBAC.
5. Validate every API input.
6. Business logic belongs in services/domain layers.
7. Controllers must remain thin.
8. Every state-changing action must be auditable.
9. Do not access another module's database tables directly.
10. Use domain events for cross-module communication.
11. Write tests for all critical business rules.
12. Do not introduce new dependencies without approval.
13. Do not modify unrelated modules.
14. Preserve backward API compatibility.
15. Follow repository naming conventions.
16. Prefer reusable components.
17. Use transactions for multi-step critical operations.
18. All errors must use the standard API error structure.

26. CODEX.md Master Instructions

Before writing code:

1. Read CLAUDE.md.
2. Read the relevant module specification.
3. Identify affected database entities.
4. Identify affected APIs.
5. Identify permission requirements.
6. Identify cross-module events.

7. Create an implementation plan.

Implementation sequence:

1. Database schema
2. Migration
3. Domain entities
4. DTOs and validation
5. Repository
6. Service
7. API controller
8. Permissions
9. Events
10. Tests
11. API documentation

After implementation:

1. Run tests.
2. Check tenant isolation.
3. Check authorization.
4. Check validation.
5. Check audit logging.
6. Check error handling.
7. Check API documentation.

27. AI Development Workflow

Every module shall follow this workflow.

Step 1: Understand

AI reads:

- Product specification
- Module specification
- Database schema
- Existing architecture

Step 2: Plan

AI generates:

- Files to create
- Files to modify
- Database changes
- APIs
- Permissions
- Events
- Tests

Step 3: Approval

Developer reviews plan.

Step 4: Implementation

AI implements only approved scope.

Step 5: Validation

AI runs:

- Type checks
- Unit tests
- Integration tests

- Linting

28. Master Prompt for Claude Code

You are working on the CITIS Education Platform.

Read the following before making changes:

1. CLAUDE.md
2. Relevant module specification in /docs/modules
3. Database architecture in /docs/database
4. API standards in /docs/api

Your task is to implement:

[MODULE NAME]

Do not begin coding immediately.

First provide:

1. Summary of requirements understood
2. Existing files affected
3. New files required
4. Database changes
5. Migration plan
6. API endpoints

7. Permissions
8. Domain events
9. Test plan
10. Risks or ambiguities

Wait for approval before implementing.

After approval:

- Implement only the approved scope.
- Follow existing architecture.
- Maintain tenant isolation.
- Enforce RBAC.
- Add audit logging.
- Add tests.
- Update API documentation.

At completion provide:

1. Files created
2. Files modified
3. Database migrations
4. APIs created
5. Tests added
6. Events added
7. Known limitations

29. Development Phases

Phase 0: Foundation

- Monorepo
- CI/CD
- Database
- Authentication
- Multi-tenancy
- RBAC
- Audit logging
- Institution setup
- Design system

Phase 1: Core School ERP

- Student management
- Parent management
- Admission
- Academics
- Attendance
- Communication

Phase 2: Financial Operations

- Fee configuration
- Student fees
- Invoices
- Payments
- Receipts
- Parent portal

Phase 3: LMS

- Course management
- Learning content
- Student enrollment
- Assignments
- Assessments
- Progress tracking
- Certificates

Phase 4: Advanced Operations

- Online classes
- Events
- Tasks
- HRMS
- Payroll

Phase 5: Enterprise Layer

- Finance
- Advanced analytics
- Multi-campus reporting
- Enterprise integrations

Phase 6: AI Layer

- AI teacher assistant
- AI lesson planning
- AI assessment generation
- AI learning recommendations
- AI student performance insights

30. Recommended MVP

The first commercially deployable CITIS version should include:

FOUNDATION

- └— Authentication
- └— Multi-Tenant Management
- └— RBAC
- └— Institution Setup

CORE ERP

- └— Student Management
- └— Parent Management
- └— Academics
- └— Attendance
- └— Communication

LMS

- └— Courses
- └— Learning Content
- └— Assignments
- └— Assessments
- └— Progress Tracking

ENGAGEMENT

- └— Parent Portal
- └— Mobile Application

31. Definition of Done

A feature shall not be considered complete unless:

- Functional requirements are implemented
- Database migration is completed
- API validation is implemented
- Authorization is implemented
- Tenant isolation is verified
- Audit logging is implemented
- Unit tests are written
- Integration tests are completed where applicable
- API documentation is updated
- Error handling is verified
- UI is responsive
- Accessibility standards are considered
- Code review is completed
- No critical security issues remain

32. Final Product Positioning

The CITIS platform should be positioned as:

CITIS Education Cloud

One Platform. Complete Education Management.

The platform architecture shall support:

SCHOOL MANAGEMENT

+

STUDENT INFORMATION SYSTEM

+

LEARNING MANAGEMENT SYSTEM

+

DIGITAL CLASSROOM

+

PARENT ENGAGEMENT

+

INSTITUTIONAL OPERATIONS

+

AI INTELLIGENCE